

Chaque cadre = une **instance** de `fjVec`. Les étapes 4→7 ont lieu dans *chaque* instance avant qu'elle rende la main.

① `fjVec([3, 5, 18, 7])` appel principal

- 0 Straggler 6 mis de côté, `arr = [3, 5, 18, 7]`
- 1 Paires formées, grands à gauche : `arr = [5, 3, 18, 7]`
- 2 `origPairs = [(5, 3), (18, 7)]` — `larger = [5, 18]`
- 3 → appelle `fjVec(larger = [5, 18])` ↓

② `fjVec([5, 18])` appel récursif

- 0 Taille 2 = pair → *pas* de straggler
- 1 1 paire : $5 < 18$ → swap → `arr = [18, 5]`
- 2 `origPairs = [(18, 5)]` — `larger = [18]`
- 3 → appelle `fjVec(larger = [18])` ↓

③ `fjVec([18])` cas de base

— `size ≤ 1` → **return** immédiatement, rien ne change

↑ retour dans ② — `larger = [18]`, on continue :

- 4 Réassocier : `larger[0]=18` → `origPairs[0]=(18,5)` → **`sortedPairs[0]=(18,5)`**
- 5 `mainChain = [5, 18]` `pend = vide` (`m=1`, pas de `b2`)
- 6 `pend vide` → étape Jacobsthal *sautée*
- 7 pas de straggler → *sauté*
- ↩ **`arr = [5, 18]` — return (larger dans ① est maintenant [5, 18])**

↑ retour dans ① — `larger = [5, 18]`, on continue :

- 4 Réassocier :
`larger[0]=5` → `origPairs[0]=(5,3)` → `sortedPairs[0]=(5,3)`
`larger[1]=18` → `origPairs[1]=(18,7)` → `sortedPairs[1]=(18,7)`
- 5 `mainChain = [3, 5, 18]` `pend = [7]`
- 6 Jacobsthal : insérer 7 avant 18 → `mainChain = [3, 5, 7, 18]`
- 7 Straggler 6 inséré → `mainChain = [3, 5, 6, 7, 18]`
- ↩ **`arr = [3, 5, 6, 7, 18]` — terminé ✓**